

ICPC Team Reference — DP Optimizations (C++ 17)

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = 4e18; // never LLONG_MAX: a + b overflows
```

0. Which one do I need?

Recurrence you are staring at	Use	Cost
"which sums are reachable", 0/1 items	bitset (§1)	$O(n \cdot M / 64)$
same, but item w repeats c times	binary splitting (§2)	$O(M / 64 \cdot \sum \log c)$
same, need no $\log$ / need reconstruction	counter trick (§2b)	$O(M)$ per distinct w
$\sum a_i \leq N$ (sum bounded, not values)	only $O(\sqrt{N})$ distinct values (§3)	$O(N\sqrt{N} / 64)$
$dp[i][j] = \min_k dp[i-1][k-1] + C(k, j)$	D&C DP (§4)	$O(m \cdot n \cdot \log n)$
$dp[i][j] = \min_k dp[i][k] + dp[k+1][j] + C(i, j)$	Knuth (§5)	$O(n^2)$
$dp[i] = \min_j dp[j] + b_j \cdot x_i$ (cost = product)	Li Chao / CHT (§6b)	$O(n \log n)$
"exactly k groups", $F(k)$ convex	Aliens / WQS (§6c)	kills the k dimension
$f[\text{mask}] = \sum g[\text{sub}], \text{sub} \subseteq \text{mask}$	SOS DP (§6d)	$O(2^B \cdot B)$

Read this before betting a submission on §4/§5: both need the *quadrangle inequality*. Do **not** prove it at the whiteboard during a contest — brute-force check it (§6a). It takes 90 seconds and it is the single highest-value thing on this page.

1. Bitset subset sum (0/1 items)

When. "Is sum x reachable / print all reachable sums", weights up to M , n items, no counting needed (feasibility only, not "in how many ways"). Kills a factor of 64 off the classic $O(n \cdot M)$ knapsack.

Complexity. $O(n \cdot M / 64)$. $n = M = 10^5 \rightarrow \sim 1.5 \cdot 10^8 / 64 \approx$ trivial.

```
const int M = 100005; // must be > max reachable sum, compile-time
bitset<M> bs;
```

```
void init() { bs.reset(); bs[0] = 1; } // sum 0 always reachable (take nothing)
void add(int w) { bs |= bs << w; } // one copy of an item of weight w

// bs[x] == 1 <=> sum x is reachable.
// Walking every reachable sum in O(answers + M/64):
// for (int x = bs._Find_first(); x < M; x = bs._Find_next(x)) { ... }
```

Notes.

- `bs << s` with `s >= M` is well-defined and gives all zeros — no UB, but guard `1LL*w*cur` against `int` overflow before shifting (§2).
- Need "reachable using exactly `k` items"? Keep `bitset<M> f[K+1]` and update `k` **descending**: `for (int k = K-1; k >= 0; k--) f[k+1] |= f[k] << w;`
- Need to *reconstruct* the chosen subset? A bitset forgets everything. Either redo one $O(n \cdot M)$ pass storing `who[x]`, or use §2b which carries counters, or D&C over the item list.

2. Bounded knapsack — binary splitting

When. Items come as (weight `w`, count `c`) and `c` is large. Naively pushing `c` copies is $O(c)$ shifts. Split `c` into powers of two: any $k \in [0, c]$ is a subset sum of $\{1, 2, 4, \dots, \text{rest}\}$, so `c` groups collapse into $O(\log c)$ 0/1 items. `c = 13 → 1+2+4+6`.

Complexity. $O(M/64 \cdot \sum \log c_i)$ instead of $O(M/64 \cdot \sum c_i)$.

```
// mp : weight -> count
for (auto [w, c] : mp)
{
    for (int take = 1; c > 0; take <= 1)
    {
        int cur = min(take, c); // group sizes
        1,2,4,...,rest
        if (1LL * w * cur < M) bs |= bs << (w * cur); // skip, do NOT break
        c -= cur;
    }
}
```

Trap. The *last* group can be smaller than the previous one (`c = 8` gives `1,2,4,1`), so a `break` when `w*cur >= M` silently drops a useful small group. Skip, never break.

2b. The same thing without the log (counter trick)

When. $\sum \log c$ is not small, or you need reconstruction, or `M` is a runtime value so `bitset` is awkward. $O(M)$ per **distinct** weight — no `log`, but you also lose the `/64`, so `bitset` still wins unless $\log c > 64$ -ish. Its real value is that it remembers *how many copies* were used.

```
// dp[j] : j reachable ; rem[j] : copies of the CURRENT item still unused at j
vector<char> dp(S + 1, 0);
```

```

vector<int> rem(S + 1);
dp[0] = 1;
for (auto [w, c] : mp)
{
    fill(rem.begin(), rem.end(), 0);
    for (int j = 0; j <= S; j++)
    {
        if (dp[j]) rem[j] = c; // reached without this
item
        else if (j >= w && rem[j - w] > 0)
            dp[j] = 1, rem[j] = rem[j - w] - 1;
    }
}

```

Ascending **j**, and **rem** **must** be cleared per item.

2c. Runtime-sized bitset (raw words)

When **M** is only known at runtime, or you want **bs** |= **bs** << **s** on a slice:

```

// b : n64 = (M+63)/64 words. Does b |= b << s.
void shift_or(uint64_t *b, int n64, int s)
{
    int w = s >> 6, r = s & 63;
    if (r == 0)
        for (int i = n64 - 1; i >= w; i--) b[i] |= b[i - w];
    else
        for (int i = n64 - 1; i >= w; i--)
        {
            uint64_t x = b[i - w] << r;
            if (i - w - 1 >= 0) x |= b[i - w - 1] >> (64 - r);
            b[i] |= x;
        }
}

```

Descending **i** is mandatory (we read words below the one we write).

3. $\sum a_i \leq N \Rightarrow$ only $O(\sqrt{N})$ distinct values

When. The statement bounds the **sum**, not the values: " $n \leq 2 \cdot 10^5$, $\sum a_i \leq 2 \cdot 10^5$, which sums are achievable / can we hit **X** / split into two halves". This is the single most missed observation in subset-sum problems.

Lemma. All $a_i \geq 1$ and $\sum a_i = S \Rightarrow$ at most $\sqrt{2S}$ **distinct** values (having **d** distinct values forces $S \geq 1+2+\dots+d$).

So: compress to (value, count) and you have $O(\sqrt{S})$ items — then §2 or §2b.

```
map<int, int> mp; // |mp| = O(sqrt(S))
for (int x : a) mp[x]++;
// now run §2 (bitset + binary splitting) or §2b over mp
```

Complexity. $O(S\sqrt{S})$ with §2b, $O(S \cdot \sqrt{S} \cdot \log/64)$ with §2 — for $S = 10^5$ that is ~450 distinct values, i.e. instant either way. Same lemma powers "number of distinct subset sums of a multiset with bounded total".

Related trick. To track sum *and* count together, use the combined weight $w \cdot (n+1) + 1$ in one bitset — only if $S \cdot (n+1)$ fits in memory.

4. Divide & Conquer DP

When.

```
dp[i][j] = min over 0 <= k <= j of ( dp[i-1][k-1] + C(k, j) ),    dp[i][j<0] = 0
```

m layers $\times$ n columns, C in $O(1)$. Requires $\text{opt}(i, j) \leq \text{opt}(i, j+1)$ (monotone optimal split), guaranteed by the **quadrangle inequality**

```
C(a, c) + C(b, d) <= C(a, d) + C(b, c)    for all a <= b <= c <= d
```

Complexity. $O(m \cdot n \cdot \log n)$ down from $O(m \cdot n^2)$.

```
int n, m;
vector<ll> cur, prv;
ll C(int a, int b); // cost of segment [a, b], O(1)

// fill cur[l..r] knowing the optimal split lies inside [optl, opttr]
void rec(int l, int r, int optl, int opttr)
{
    if (l > r) return;
    int mid = (l + r) >> 1, opt = optl;
    ll best = INF;
    for (int k = optl; k <= min(mid, opttr); k++)
    {
        ll val = (k ? prv[k - 1] : 0) + C(k, mid);
        if (val < best) best = val, opt = k;
    }
    cur[mid] = best;
    rec(l, mid - 1, optl, opt);
    rec(mid + 1, r, opt, opttr);
}

ll solve()
{

```

```

    prv.assign(n, 0), cur.assign(n, 0);
    for (int j = 0; j < n; j++) prv[j] = C(0, j);    // layer 0
    for (int i = 1; i < m; i++)
    {
        rec(0, n - 1, 0, n - 1);
        swap(prv, cur);                            // swap, not copy
    }
    return prv[n - 1];
}

```

Trap. D&C DP needs the **previous** layer to be complete. If $dp[j]$ depends on $dp[k]$ in the *same* layer, this is wrong — that is the 1D/1D case, use CHT (§6b) or a monotone stack of candidate intervals.

4b. When $C(1, r)$ is not $O(1)$

Classic in "split into m parts minimising Σ pairs of equal elements" (CF 868F). Keep a Mo's-style window with *add/del* and let the D&C move the pointers — total $O(n \log n)$ moves per layer.

```

int L = 0, R = -1; ll cur_cost = 0;
void addR(int i); void addL(int i); void delR(int i); void delL(int i);

ll C(int l, int r)                // EXPAND first, then shrink
{
    while (R < r) addR(++R);
    while (L > l) addL(--L);
    while (R > r) delR(R--);
    while (L < l) delL(L++);
    return cur_cost;
}

```

5. Knuth's optimization (Knuth–Yao)

When. Range DP / interval merging:

$$dp[i][j] = \min_{i \leq k < j} (dp[i][k] + dp[k+1][j] + C(i, j))$$

Applies when $opt(i, j-1) \leq opt(i, j) \leq opt(i+1, j)$, guaranteed by

1. **monotone cost:** $C(b, c) \leq C(a, d)$ for $a \leq b \leq c \leq d$
2. **quadrangle inequality:** $C(a, c) + C(b, d) \leq C(a, d) + C(b, c)$

Both hold trivially when $C(i, j)$ is a subarray sum of non-negative values — the usual case (optimal BST, merging stones, Cutting Sticks, Breaking String).

Complexity. $O(n^2)$ instead of $O(n^3)$. The window sizes $opt(i+1, j) - opt(i, j-1)$ telescope, so the whole table costs $O(n^2)$.

```

const int N = 2005;
ll dp[N][N]; int opt[N][N];
int n;
ll C(int i, int j);

ll knuth()
{
    for (int i = 0; i < n; i++)
        dp[i][i] = 0, opt[i][i] = i;           // base case, problem-specific

    for (int i = n - 2; i >= 0; i--)           // i descending
        for (int j = i + 1; j < n; j++)       // j ascending
        {
            ll best = INF, c = C(i, j);
            int lo = opt[i][j - 1], hi = min(j - 1, opt[i + 1][j]);
            for (int k = lo; k <= hi; k++)
            {
                ll val = dp[i][k] + dp[k + 1][j] + c;
                if (val <= best) best = val, opt[i][j] = k;    // <= : keep LARGEST
            }
            dp[i][j] = best;
        }
    return dp[0][n - 1];
}

```

Traps.

- Iteration order is not negotiable: `dp[i][j-1]` and `dp[i+1][j]` must exist before `dp[i][j]`. `i` down, `j` up.
- Use `<=` when updating `opt`, so `opt` is the **largest** minimiser; with `<` the `opt[i][j-1] <= opt[i][j]` bound can break on ties.
- Memory: two $n \times n$ tables. `n = 5000` is ~ 300 MB $\rightarrow$ dead. `n  $\leq$  2000–3000` only; shrink `opt` to `int` / `short` if tight.

6. Things you will wish you had printed

6a. Brute-force verifier for QI / monotone opt — *use this*

You will never prove the quadrangle inequality under contest pressure. Test it.

```

// 1) does the cost obey the inequalities?
for (int a = 0; a < n; a++)
    for (int b = a; b < n; b++)
        for (int c = b; c < n; c++)
            for (int d = c; d < n; d++)
            {
                assert(C(a,c) + C(b,d) <= C(a,d) + C(b,c));    // QI (D&C + Knuth)
                assert(C(b,c) <= C(a,d));                     // monotone (Knuth only)
            }

```

```

    }
    // 2) better: run the honest  $O(n^3)$  /  $O(m \cdot n^2)$  DP for  $n \leq 60$  on random inputs
    //    and diff it against the fast one. If they match on 1000 random tests,
    //    submit. This is faster than a proof and strictly more convincing.

```

6b. Li Chao tree — when the cost is a product

If $C(j+1, i)$ can be written as $b_j \cdot x_i + \text{const}$, then $\text{dp}[i] = \min_j (\text{dp}[j] + b_j \cdot x_i)$ is a lower-envelope query: **$O(n \log n)$, beats D&C, and needs no monotonicity proof.** Li Chao over monotone CHT because it does not care about the insertion order of slopes.

```

struct Line
{
    ll m = 0, b = INF;
    ll operator()(ll x) const { return m * x + b; }
};

struct LiChao // minimum over lines, x in [0, n)
{
    int n; vector<Line> t;
    LiChao(int n) : n(n), t(4 * n) {}
    void add(Line ln, int v, int l, int r)
    {
        while (true)
        {
            int m = (l + r) >> 1;
            bool lf = ln(l) < t[v](l), md = ln(m) < t[v](m); // BEFORE the swap
            if (md) swap(t[v], ln);
            if (l == r) return;
            if (lf != md) v = 2 * v, r = m;
            else v = 2 * v + 1, l = m + 1;
        }
    }
    void add(ll m, ll b) { add({m, b}, 1, 0, n - 1); }
    ll query(int x)
    {
        ll res = INF; int v = 1, l = 0, r = n - 1;
        while (true)
        {
            res = min(res, t[v](x));
            if (l == r) return res;
            int m = (l + r) >> 1;
            if (x <= m) v = 2 * v, r = m;
            else v = 2 * v + 1, l = m + 1;
        }
    }
};

// maximum instead: insert negated lines and negate the answer.
// x not an index? compress, or template the tree over [lo,hi] with ll midpoints.

```

6c. Aliens trick (WQS binary search) — deletes the "exactly k" dimension

When. $F(k)$ = best cost using **exactly** k groups/segments/edges, and F is **convex** in k (test it: brute force small n , check second differences). Charge a penalty λ per group, solve the *unconstrained* problem (which is usually §4/§6b without the k layer), and binary search λ .

Complexity. $O(\text{cost_of_one_solve} \cdot \log(\max \text{ cost}))$. Turns $O(k \cdot n \cdot \log n)$ into $O(n \log n \log C)$.

```
// solve(lam) -> { min over j of (F(j) + lam*j), minimum j attaining it }
pair<ll, int> solve(ll lam);

ll aliens(int k, ll LO, ll HI)          // LO..HI = plausible range of lambda
{
    ll lo = LO, hi = HI, lam = HI;
    while (lo <= hi)
    {
        ll mid = lo + (hi - lo) / 2;
        if (solve(mid).second <= k) lam = mid, hi = mid - 1;
        else lo = mid + 1;
    }
    return solve(lam).first - lam * k;
}
```

Traps. Break ties toward the **minimum** group count inside `solve`, or the $\leq k$ test lies. Integer λ is enough when costs are integers. Convexity is a real precondition — if F is not convex the answer is silently wrong.

6d. SOS DP (sum over subsets)

```
for (int i = 0; i < B; i++)                // f[mask] = sum of f[sub], sub ⊆
mask                                       mask
    for (int m = 0; m < (1 << B); m++)
        if (m >> i & 1) f[m] += f[m ^ (1 << i)];
// supersets: swap the condition to if (!(m >> i & 1)) f[m] += f[m | (1 << i)];
// inverse (Möbius): same loops with -= , iterate identically.
```

$O(2^B \cdot B)$. Use for "count pairs with $a \& b == 0$ ", "for every mask, best value over its submasks".

6e. Slope trick (one-liner version)

Minimum total cost to make a non-decreasing, cost $|change|$:

```
priority_queue<ll> pq; ll ans = 0;
for (ll x : a)
{
    pq.push(x);
    if (pq.top() > x) { ans += pq.top() - x; pq.pop(); pq.push(x); }
}
```

$O(n \log n)$. Generalises to convex piecewise-linear **dp** functions maintained as a heap of slope-change points.

7. Contest checklist

- 1. **Complexity first.** $n \leq 5000$ and $O(n^2)$ → just write the n^2 . These optimizations are for $n \geq 10^5$ or n^2 layers, not for showing off.
- 2. **INF = 4e18 is wrong too if you add two of them.** Use **4e18** for **11** comparisons, **1e18** if you ever add. Never **LLONG_MAX**.
- 3. Clear **globals** between test cases: **bs**, **opt**, the D&C pointers **L/R**, the Mo's structure. Multi-test is where these templates die.
- 4. **bitset<M>** needs a compile-time **M** — size it to the max sum **plus one**.
- 5. D&C DP: previous layer only. Knuth: **i** down, **j** up, **<=** on ties.
- 6. Before trusting QI, run §6a. Before trusting Aliens, check convexity.
- 7. If the cost is a subarray sum, you almost certainly have QI for free — Knuth and D&C are both safe.
- 8. Stack depth: **rec** is $O(\log n)$ deep, fine. The $O(n)$ -deep recursions are the ones that TLE/segfault.

8. Cost functions that are known to satisfy QI

Recognising these on sight saves the proof entirely. Let **pre** be a prefix-sum array over **non-negative** values and $s(i,j) = pre[j+1] - pre[i]$.

c(i,j)	QI	monotone	notes
$s(i,j)$ (plain subarray sum)	yes (equality)	yes	merging stones, Cutting Sticks — Knuth-safe
$f(s(i,j))$, f convex non-decreasing	yes	yes	covers s^2 , $s \cdot \log s$, e^s
#pairs of equal elements inside $[i,j]$	yes	yes	CF 868F, use §4b for the cost
$\sum_{t \in [i,j]} (t - i) \cdot a[t]$ (weighted by offset)	yes	yes	"Circular Barn"-style
$\sum p_t \cdot depth_t$ (optimal BST)	yes	yes	Knuth's original problem
anything with negative $a[t]$	check it	check it	negatives break monotonicity first

Key closure property: if **f** is convex and $a[t] \geq 0$, then $f(s(i,j))$ satisfies QI. That single line covers the large majority of contest cost functions. Sums of QI functions are QI, so you can add pieces freely.

9. Classic problems by shape (recognition drill)

Statement smells like	Technique
"split the array into exactly k contiguous parts, minimise $\sum f(part)$ "	§4 D&C; if k is huge, §6c Aliens

Statement smells like	Technique
"...and $f(\text{part})$ is $(\text{sum})^2$ or similar convex"	§4, or §6b if it factors into a product
"merge n piles, cost = combined size"	§5 Knuth ($O(n^2)$)
"cut a stick at given points, cost = current length"	§5 Knuth
"build a BST minimising expected search cost"	§5 Knuth
"pick a subset summing to X , n up to 10^5 , values large but Σ small"	§3 + §2
"coins with multiplicities, which totals are reachable"	§2 (or §2b if you must print the coins used)
"min cost to make the array non-decreasing"	§6e slope trick
"for each mask, aggregate over its submasks"	§6d SOS
" $dp[i] = \min(dp[j] + (i-j) \cdot \text{something}_j)$ "	§6b Li Chao / CHT

Sources: *cp-algorithms.com* (Divide & Conquer DP, Knuth's Optimization, CC BY-SA 4.0); F. F. Yao, "Efficient Dynamic Programming Using Quadrangle Inequalities" (1980). Every snippet here was compiled with *-Wall -Wextra* and stress-tested against a brute force before printing.